



Indexing and Hashing



OUTLINE

- ✓ Indexing: Basic Concepts
- ✓ Evaluation Factors
- ✓ Ordered Indices: Primary and Secondary
- ✓ Dense and Sparse indices
- ✓ Multilevel Indexing
- ✓ B+ Tree Index Files
- ✓ B-Tree Index Files
- ✓ Hashing
- ✓ Hash File Organization
- ✓ Handling of Bucket Overflows
- ✓ Open and Closed hashing
- ✓ Hash Indices



Indexing and Hashing

❑ Database Index

A data structure that improves the speed of data retrieval operations on a database table at the cost of slower writes and the use of more storage space.

❑ Basic Concept

An index for a file in a database system works in much the same way as the index in this textbook. If we want to learn about a particular topic, we can search for the topic in the index at the back of the book, find the pages where it occurs, and then read the pages to find the information we are looking for. The words in the index are in sorted order, making it easy to find the word we are looking for. Moreover, the index is much smaller than the book, further reducing the effort needed to find the words we are looking for.



Indexing and Hashing

❑ Types of Indices

There are two basic types of indices:

- **Ordered Indices**
- **Hash Indices**

Ordered Indices: Based on a sorted ordering of the values.

Hash Indices. Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a *hash function*.

❑ Evaluation Factors

There are several techniques for both ordered indexing and hashing. No one technique is the best. Rather, each technique is best suited to particular database applications. Each technique must be evaluated on the basis of the following factors:



Indexing and Hashing

❑ Evaluation Factors

- **Access Types:** Access types can include finding records with a specified attribute value and finding records whose attribute values fall in a specified range.
- **Access Time:** The time it takes to find a particular data item, or set of items, using the technique in question.
- **Insertion Time:** The time it takes to insert a new data item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.



Indexing and Hashing

❑ Evaluation Factors

- **Deletion time:** The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.
- **Space overhead:** The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.



Indexing and Hashing

❑ Search Key

An attribute or set of attributes used to look up records in a file is called a **search key**.

❑ Ordered Indices

- To gain fast random access to records in a file, we can use an index structure.
- Each index structure is associated with a particular search key.
- An ordered index stores the values of the search keys in sorted order, and associates with each search key the records that contain it.
- A file may have several indices, on different search keys.

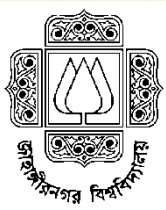


Indexing and Hashing

❑ Ordered Indices

- **Primary Index**
- **Secondary Index**

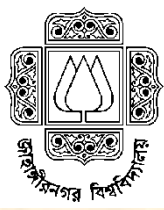
- **Primary Index:** If the file containing the records is sequentially ordered, a primary index is an index whose search key also defines the sequential order of the file.
- Primary indices are also called **clustering indices**.
- **Types:** **Dense** and **Sparse**



Indexing and Hashing

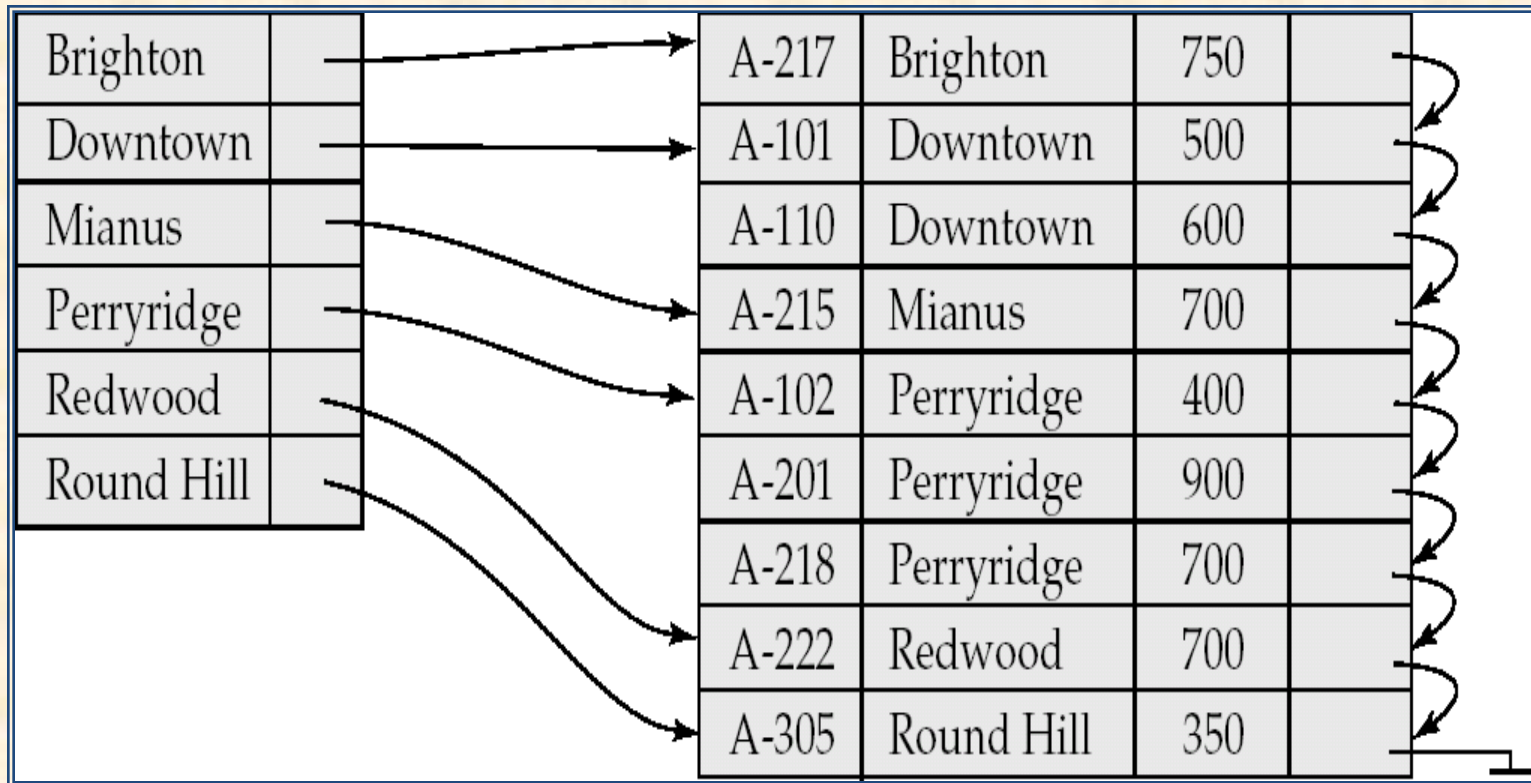
❑ Dense Index

- An index record appears for every search-key value in the file.
- In a dense primary index, the index record contains the search-key value and a pointer to the first data record with that search-key value.
- The rest of the records with the same search key-value would be stored sequentially after the first record, because the index is a primary one, records are sorted on the same search key.



Indexing and Hashing

❑ Dense Index

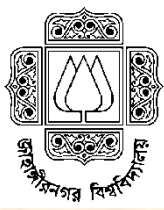




Indexing and Hashing

❑ Sparse Index

- An index record appears for only some of the search-key values.
- Each index record contains a search-key value and a pointer to the first data record with that search-key value.
- To locate a record, we find the index entry with the largest search-key value that is less than or equal to the search-key value for which we are looking.
- We start at the record pointed to by that index entry, and follow the pointers in the file until we find the desired record.



Indexing and Hashing

❑ Sparse Index

Brighton		A-217	Brighton	750	
Mianus		A-101	Downtown	500	
Redwood		A-110	Downtown	600	
		A-215	Mianus	700	
		A-102	Perryridge	400	
		A-201	Perryridge	900	
		A-218	Perryridge	700	
		A-222	Redwood	700	
		A-305	Round Hill	350	



Indexing and Hashing

❑ Dense VS Sparse Indices

- It is generally faster to locate a record if we have a dense index rather than a sparse index.
- However, sparse indices have advantages over dense indices in that they require less space and they impose less maintenance overhead for insertions and deletions.
- There is a trade-off that the system designer must make between access time and space overhead.



Indexing and Hashing

❑ Multi-Level Indices

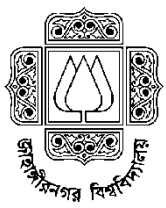
- If primary index does not fit in memory, access becomes expensive.
- Solution: treat primary index kept on disk as a sequential file and construct a sparse index on it.
 - Outer index – a sparse index of primary index
 - Inner index – the primary index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.



Indexing and Hashing

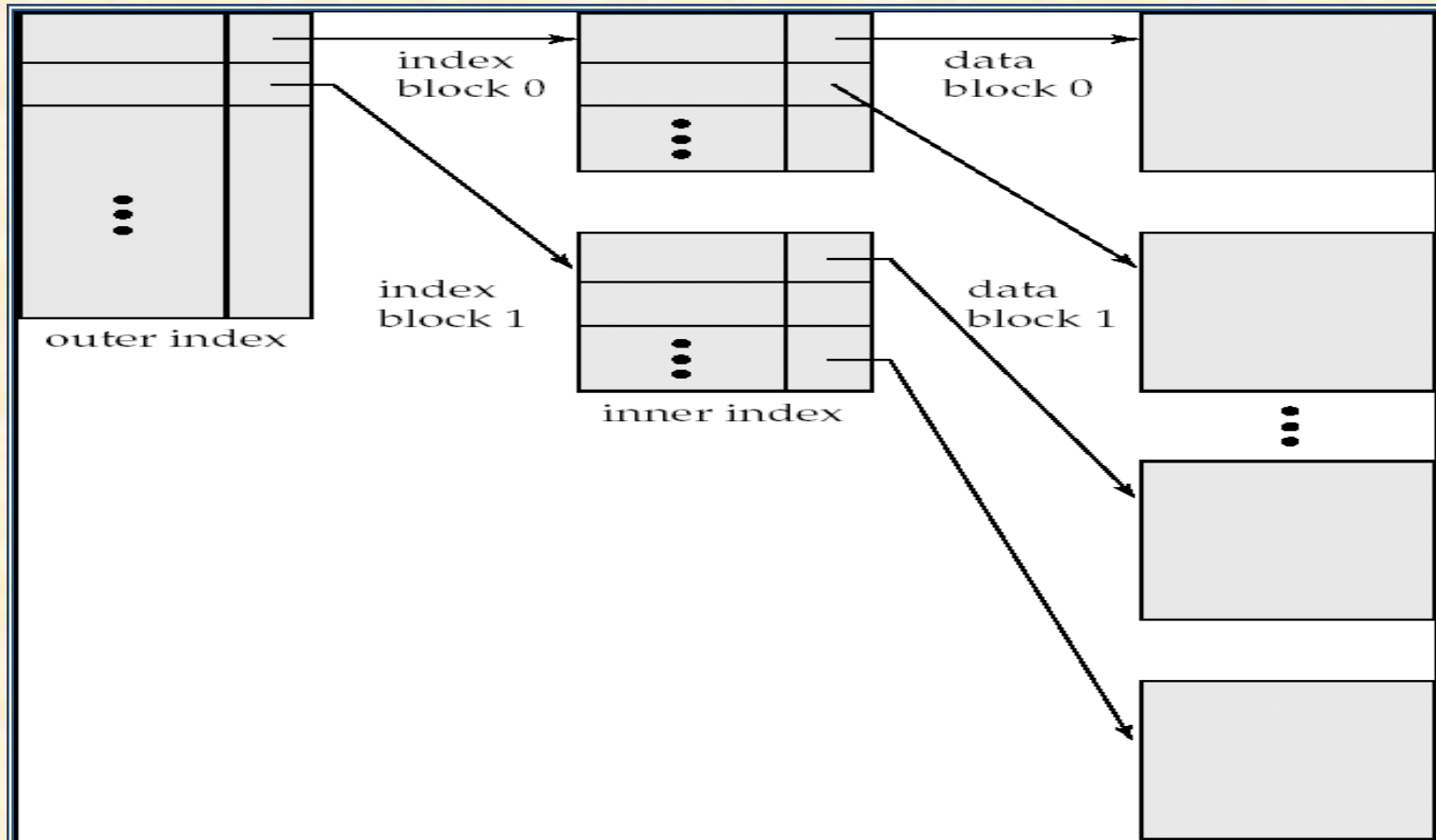
❑ Multi-Level Indices: An Example

- Consider 100,000 records, 10 per block, at one index record per block, that's 10,000 index records. Even if we can fit 100 index records per block, this is 100 blocks. If index is too large to be kept in main memory, a search results in several disk reads.
- For very large files, additional levels of indexing may be required.
- Indices must be updated at all levels when insertions or deletions require it.
- Frequently, each level of index corresponds to a unit of physical storage.



Indexing and Hashing

❑ Multi-Level Indices: An Example

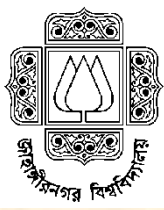




Indexing and Hashing

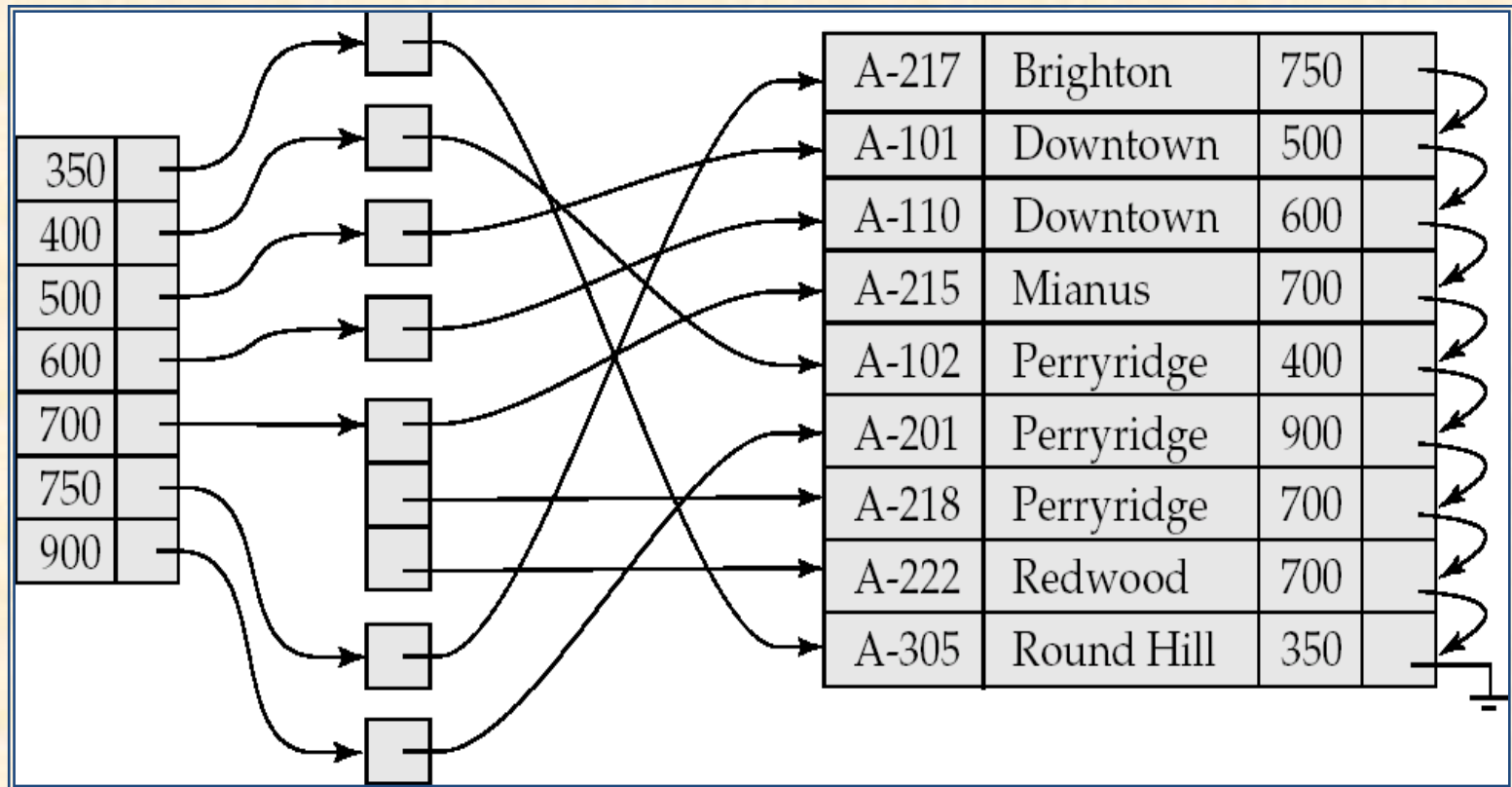
❑ Secondary Index

- Indices whose search key specifies an order different from the sequential order of the file are called **secondary indices, or non-clustering indices**.
- Secondary indices must be **dense** with an index entry for every search-key value, and a pointer to every record in the file.



Indexing and Hashing

❑ Secondary Index





Indexing and Hashing

❑ Primary VS Secondary Indices

- A sequential scan in primary index order is efficient because records in the file are stored physically in the same order as the index order.
- Secondary indices improve the performance of queries that use keys other than the search key of the primary index. However, they impose a significant overhead on modification of the database. The designer of a database decides which secondary indices are desirable on the basis of an estimate of the relative frequency of queries and modifications.
- The primary index is on the field which specifies the sequential order of the data file.
- There can be only one primary index while there can be many secondary indices.



Indexing and Hashing

❑ B+ Tree Index Files

- The main disadvantage of the index-sequential file organization is that performance degrades as the file grows, both for index lookups and for sequential scans through the data. To overcome this deficiency, we use a B⁺ tree index.
- The **B+ tree** index structure is the most widely used of several index structures that maintain their efficiency despite insertion and deletion of data.
- This is a **balanced tree** in which every path from the root of the tree to a leaf of the tree is of the same length.
- A B+ tree index is a multilevel index. A typical node of a B+tree is shown below.



Indexing and Hashing

❑ B+ Tree Index Files

- A B+ tree index is a multilevel index. A typical node of a B+-tree is shown below.



- Each node that is not a root or a leaf has between $\lceil n/2 \rceil$ and n children.
- A leaf node has between $\lceil (n-1)/2 \rceil$ and $n-1$ values
- Special cases:
 - If the root is not a leaf, it has at least 2 children.
 - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n-1)$ values.



Indexing and Hashing

❑ B+ Tree Index Files

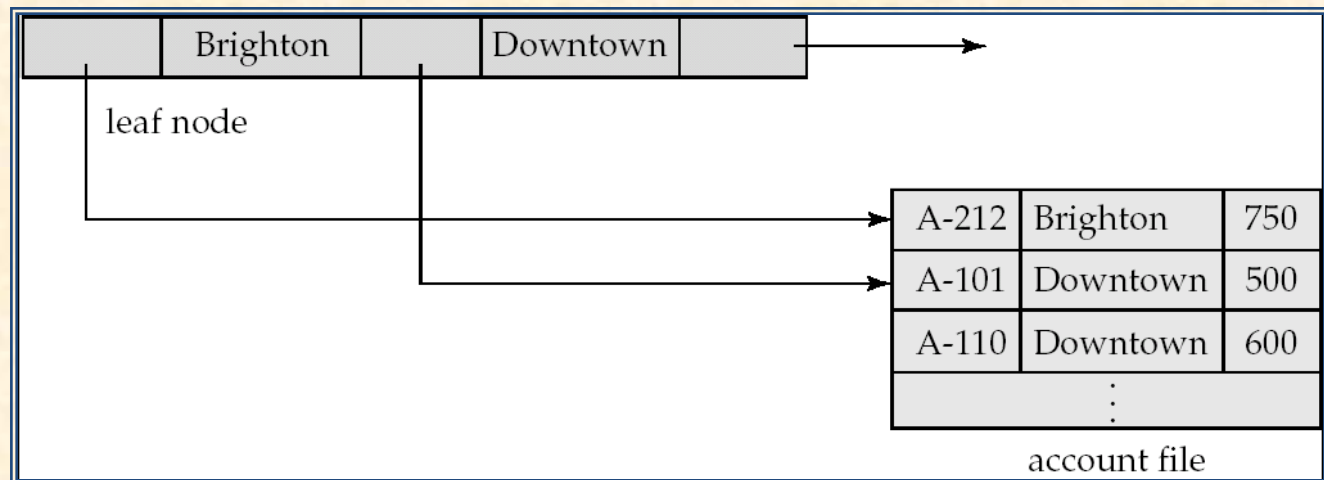
- It contains up to $n - 1$ search-key values $K_1, K_2, \dots, K_{n-1}$, and n pointers $P_1, P_2, \dots, P_n$.
- The search-keys in a node are ordered: $K_1 < K_2 < K_3 < \dots < K_{n-1}$
- For leaf nodes, for $i = 1, 2, \dots, n - 1$, pointer P_i points to either a file record with search-key value K_i or to a bucket of pointers, each of which points to a file record with search-key value K_i .



Indexing and Hashing

❑ B+ Tree Index Files

- A non-leaf node may hold up to n pointers, and must hold at least $\lceil n/2 \rceil$ pointers.
- The number of pointers in a node is called the *fanout* of the node.
- The root node can hold fewer than $\lceil n/2 \rceil$ pointers. However, it must hold at least two pointers.





Indexing and Hashing

- ❑ Construct a B+ tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

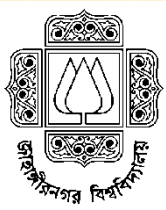
Solution: Construction of B+ tree for order $n=4$.
Search key values = 3, Pointers = 4.

Insert key value 2:

	2					
--	---	--	--	--	--	--

Insert key value 3:

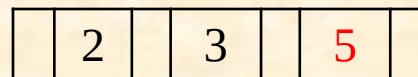
	2		3			
--	---	--	---	--	--	--



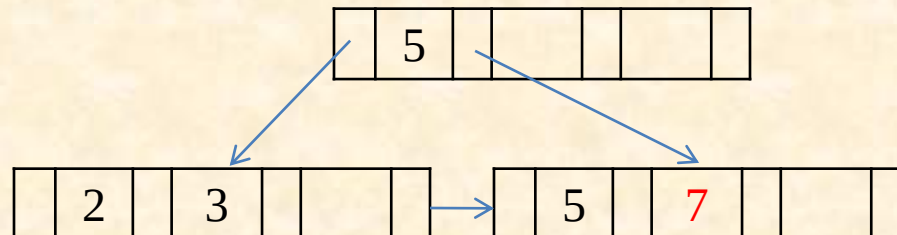
Indexing and Hashing

- ❑ Construct a B+ tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Insert key value 5:



Insert key value 7: Split the node.

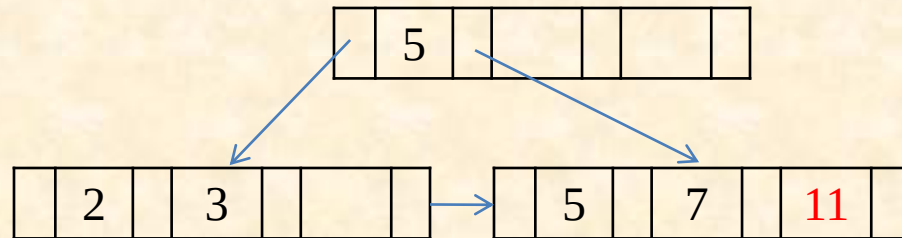




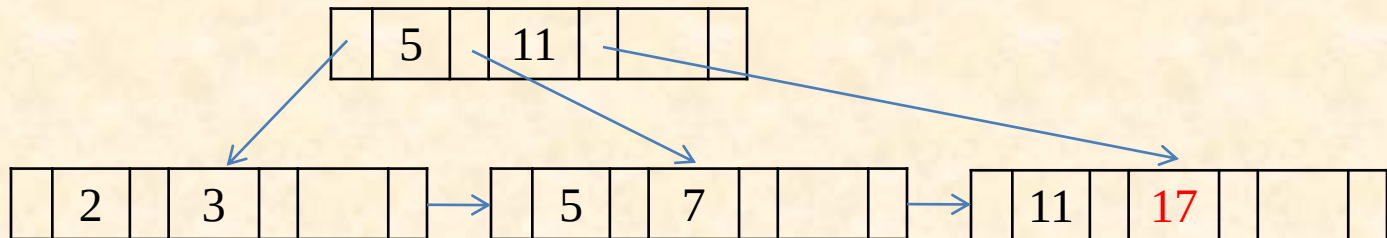
Indexing and Hashing

- ❑ Construct a B+ tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Insert key value 11:



Insert key value 17: Split the node.

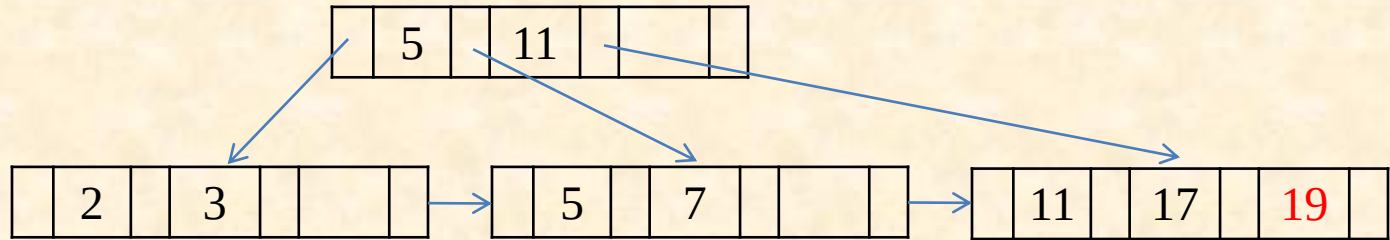




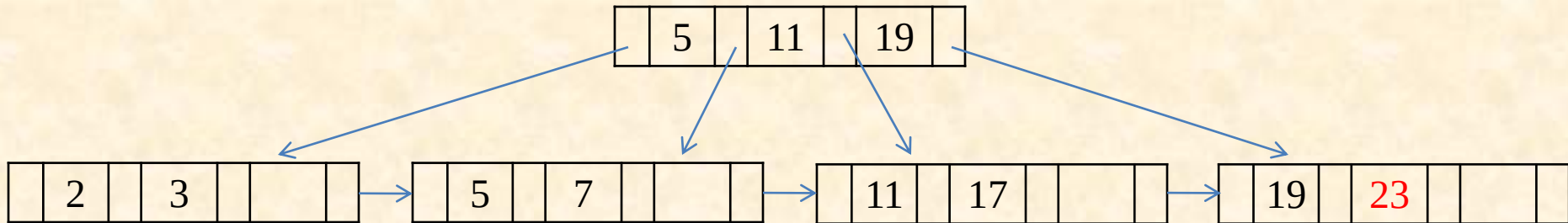
Indexing and Hashing

- ❑ Construct a B+ tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Insert key value 19:



Insert key value 23: Split the node.

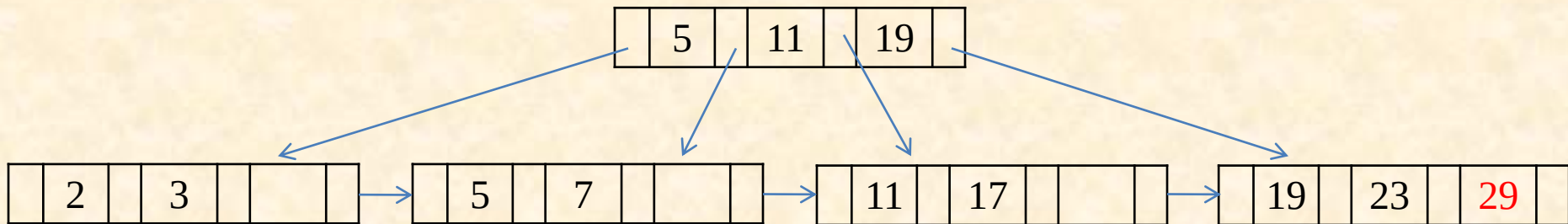




Indexing and Hashing

- ❑ Construct a B+ tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Insert key value 29:

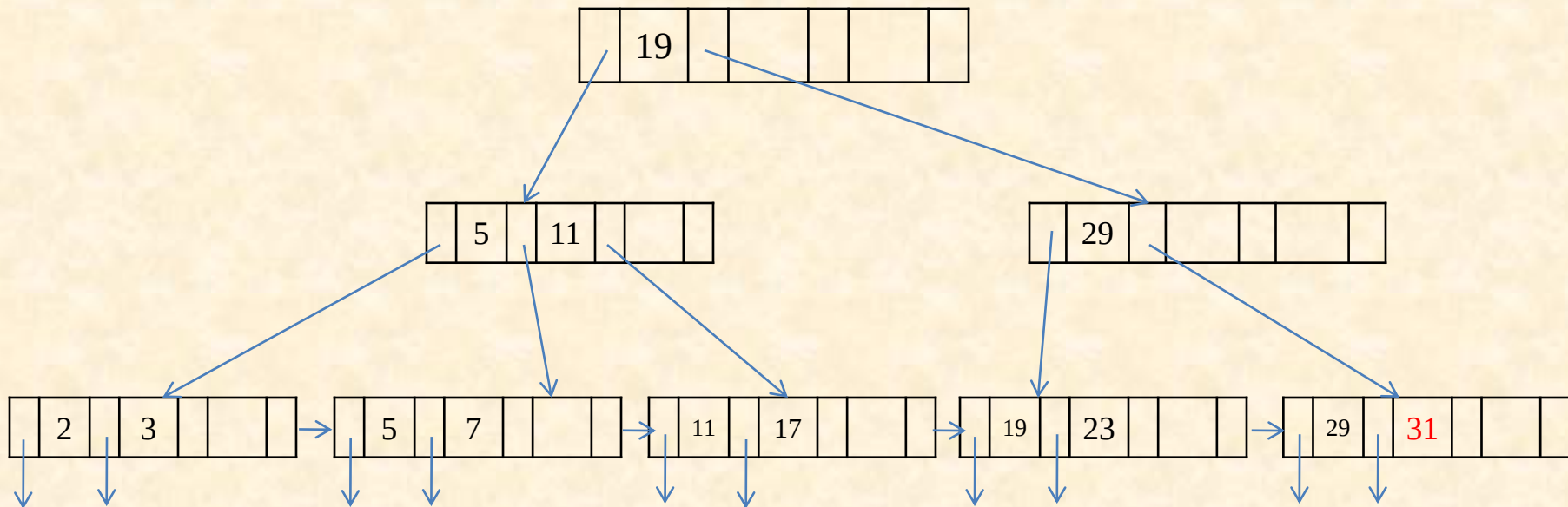




Indexing and Hashing

- ❑ Construct a B+ tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Insert key value 31:

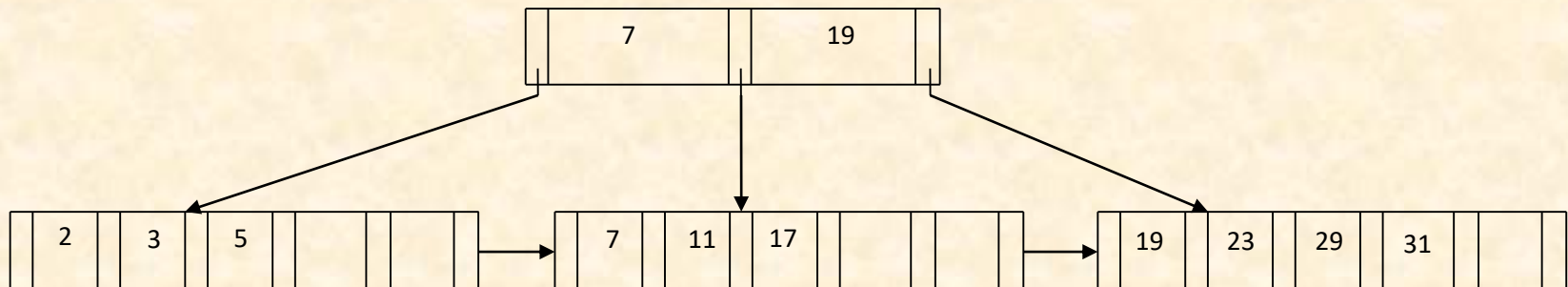




Indexing and Hashing

- ❑ Construct a B+-tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

For $n=6$:





Indexing and Hashing

❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:

5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95

b) Show how the B+ tree changes as the result of each of the following operations:

i. Insert 60

ii. Delete 65



Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: a) Construction of B+ tree for order $n=5$.
Search key values =4, Pointers= 5.

Insert key value 5, 10, 15, 20:

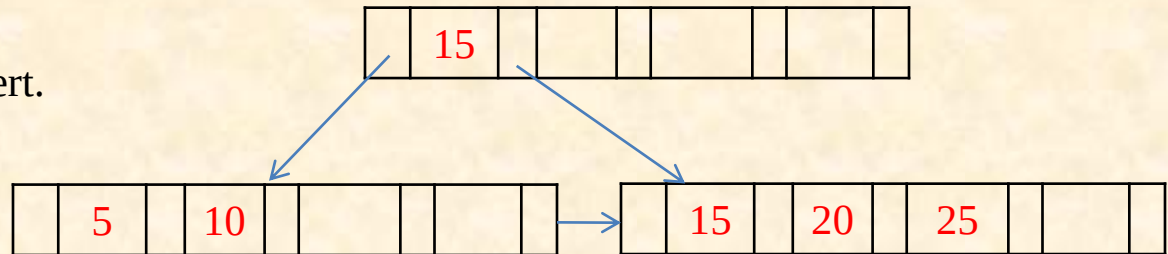
	5		10		15		20	
--	---	--	----	--	----	--	----	--



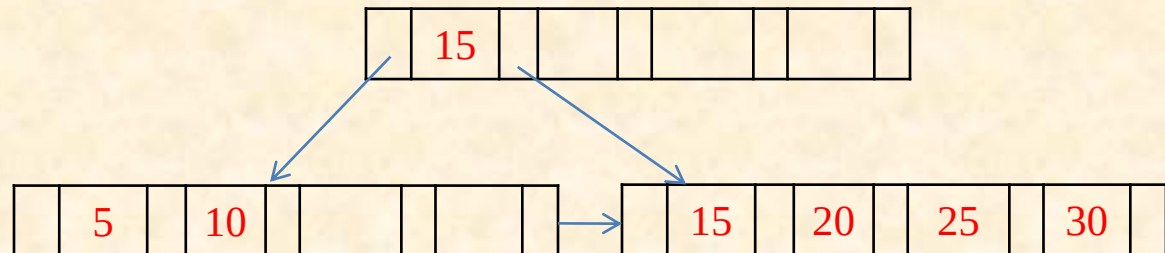
Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: Insert key value 25:
Split the node and insert.
 $\text{Ceil}((n-1)/2) = 2$



Insert key value 30:

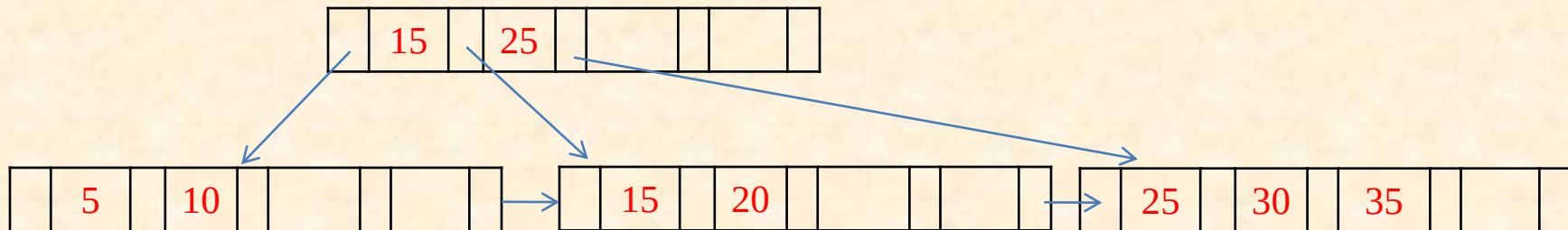




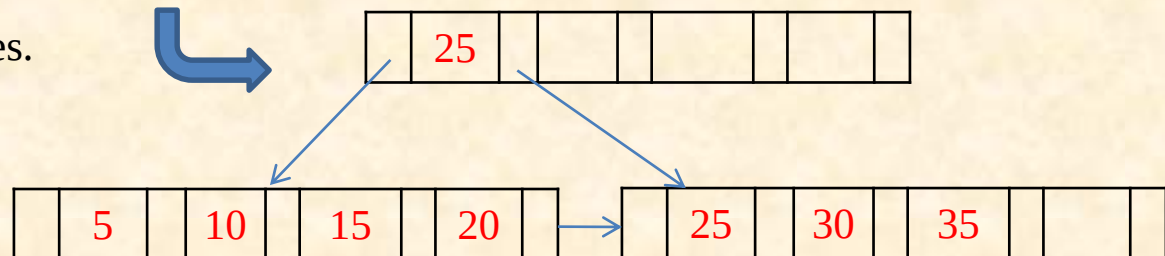
Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: Insert key value 35:
Split the node and insert.



After re-arranging nodes.

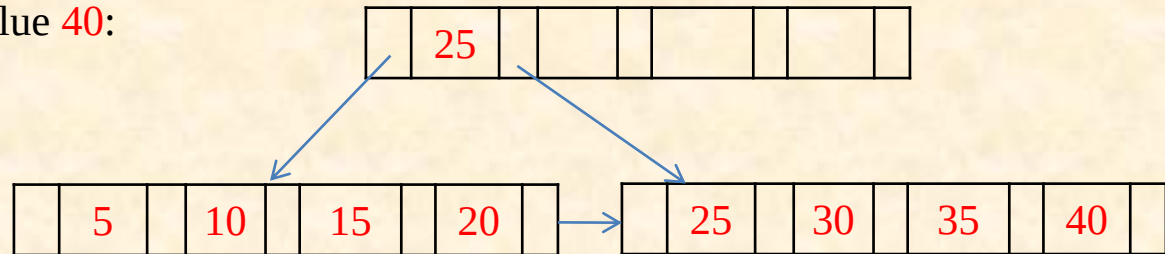




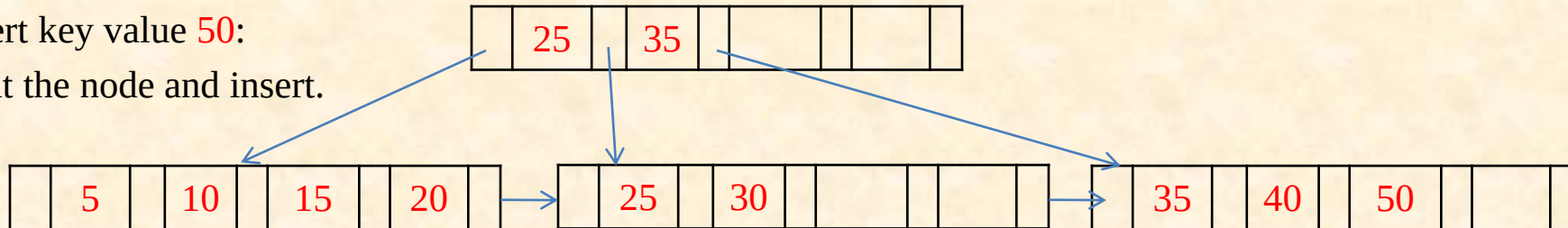
Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: Insert key value 40:



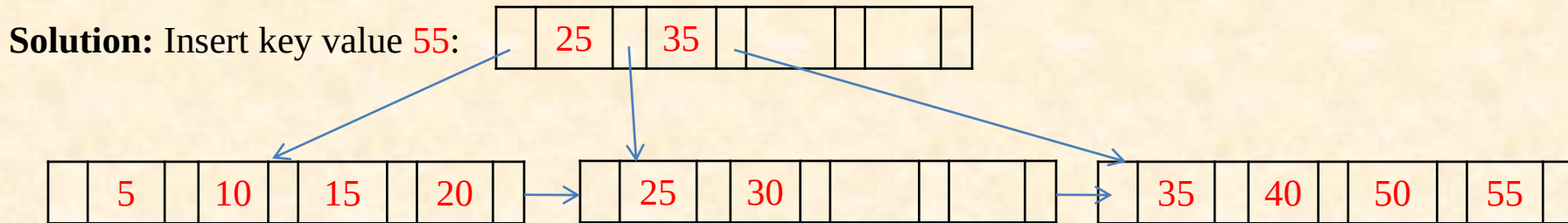
Insert key value 50:
Split the node and insert.





Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

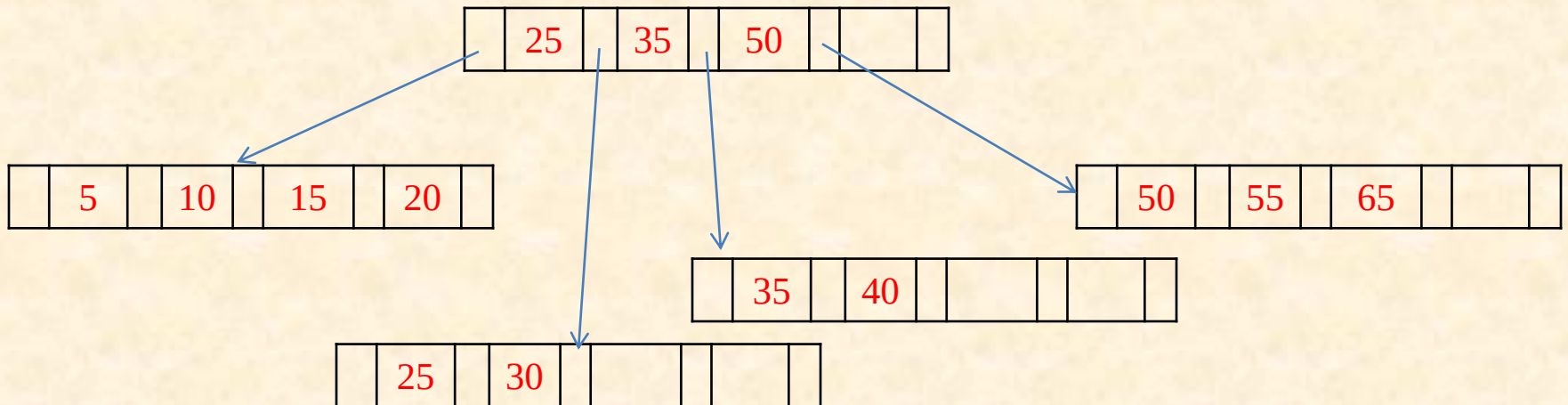




Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: Insert key value 65: Split the node and insert.

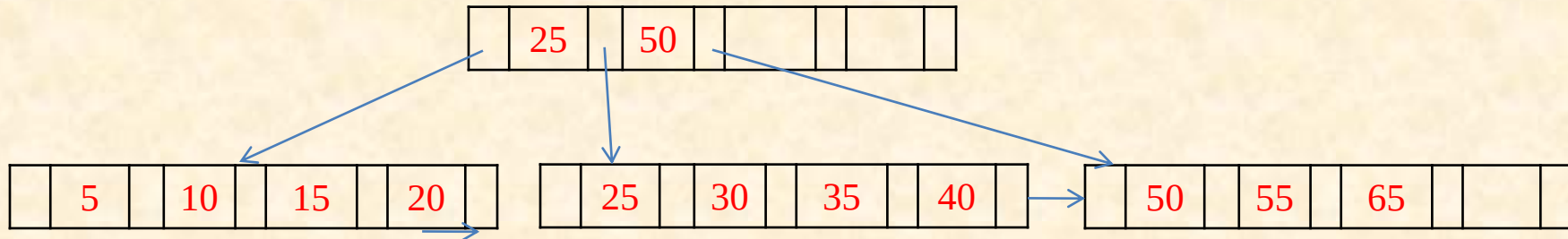




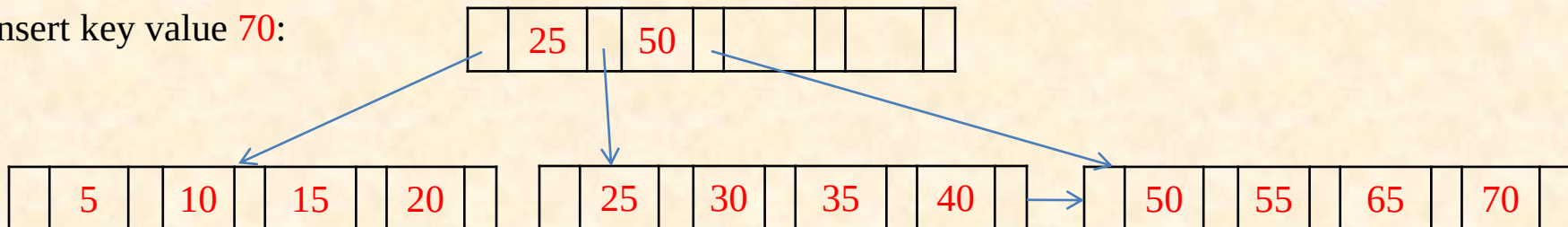
Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: After re-arranging nodes:



Insert key value 70:

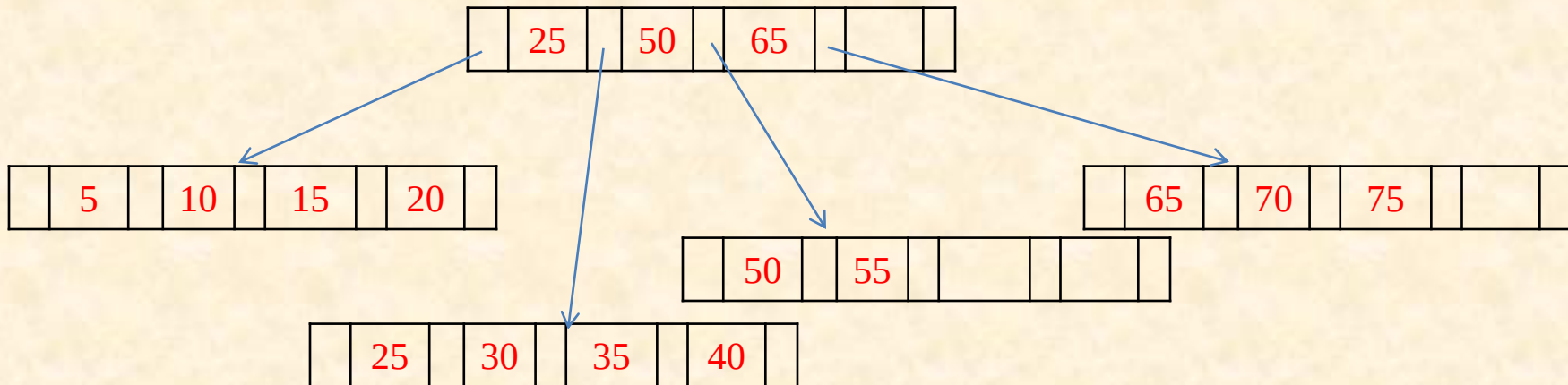




Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: Insert key value **75**: Split the node and insert.

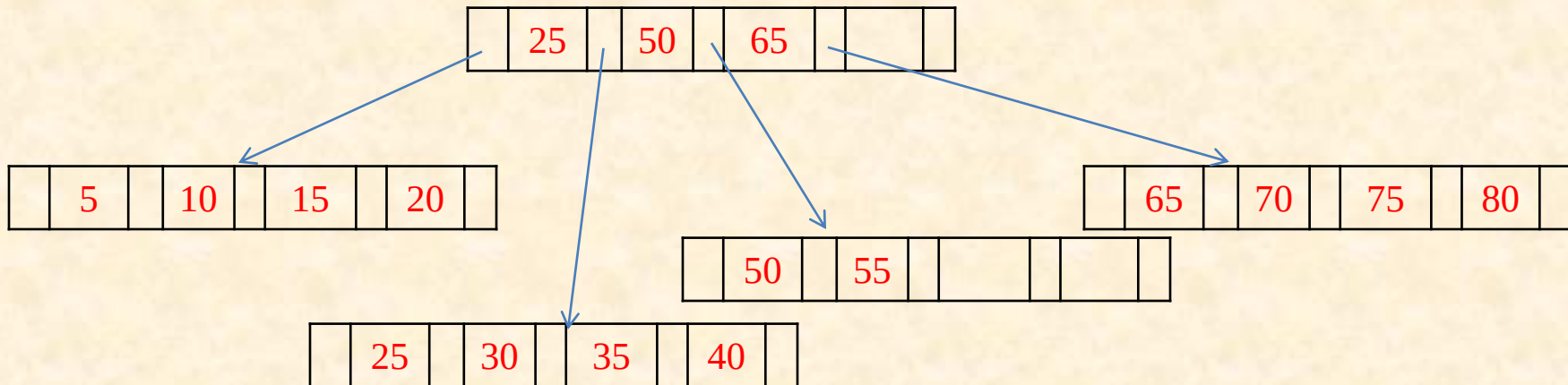




Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: Insert key value 80:

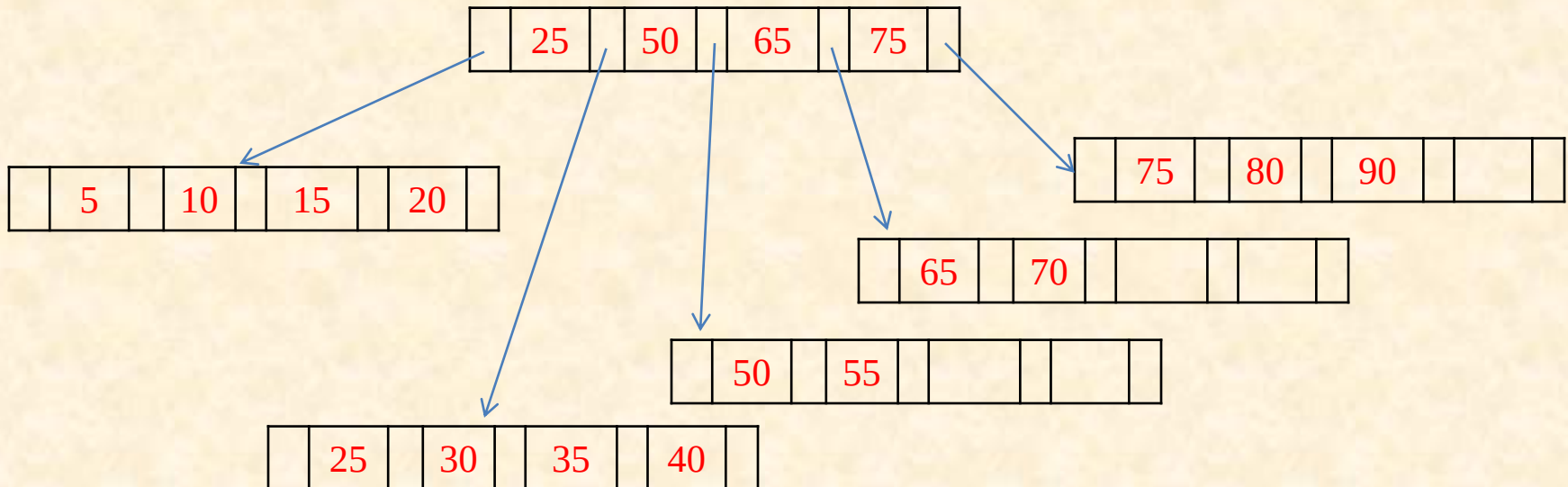




Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: Insert key value 90: Split the node and insert.

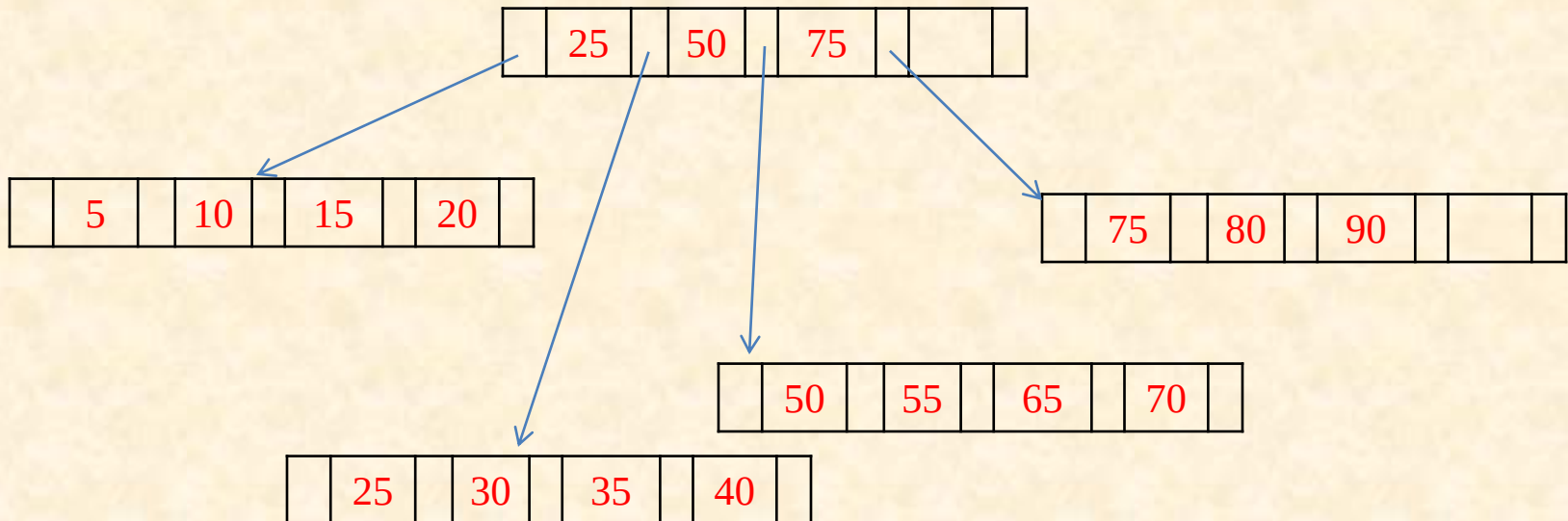




Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: After re-arranging nodes:

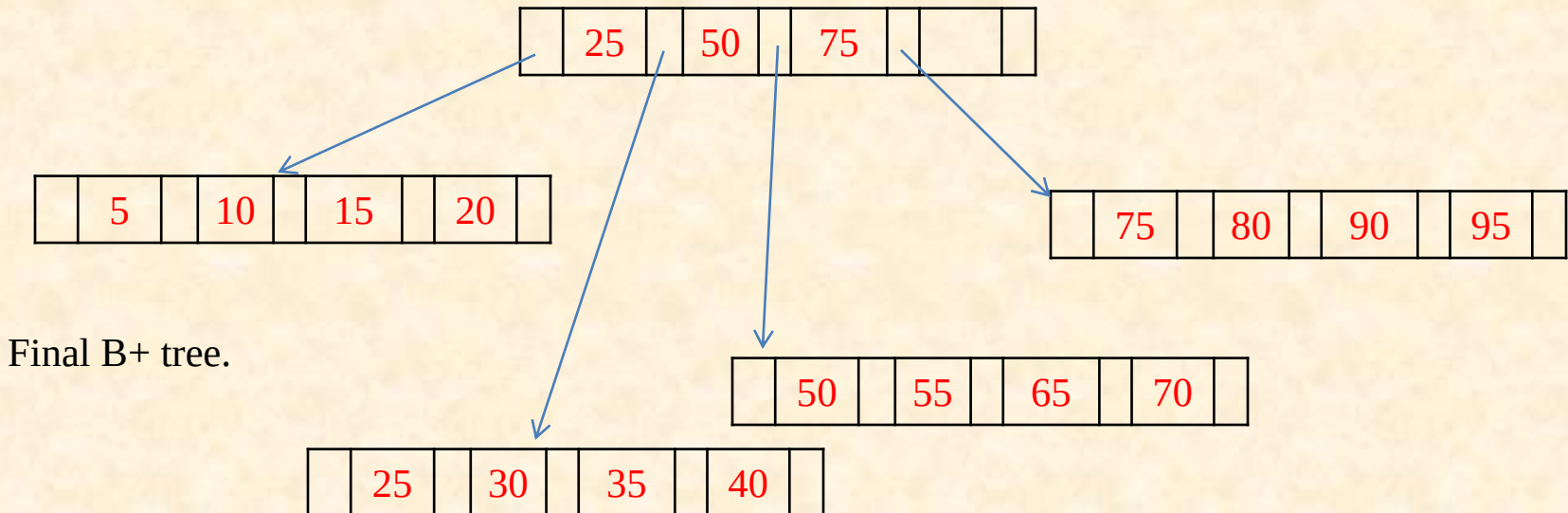




Indexing and Hashing

- ❑ **Example:** a) Construct a B+ tree for the following set of key values for $n=5$:
5, 10, 15, 20, 25, 30, 35, 40, 50, 55, 65, 70, 75, 80, 90, 95
- b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: Insert key value 95:





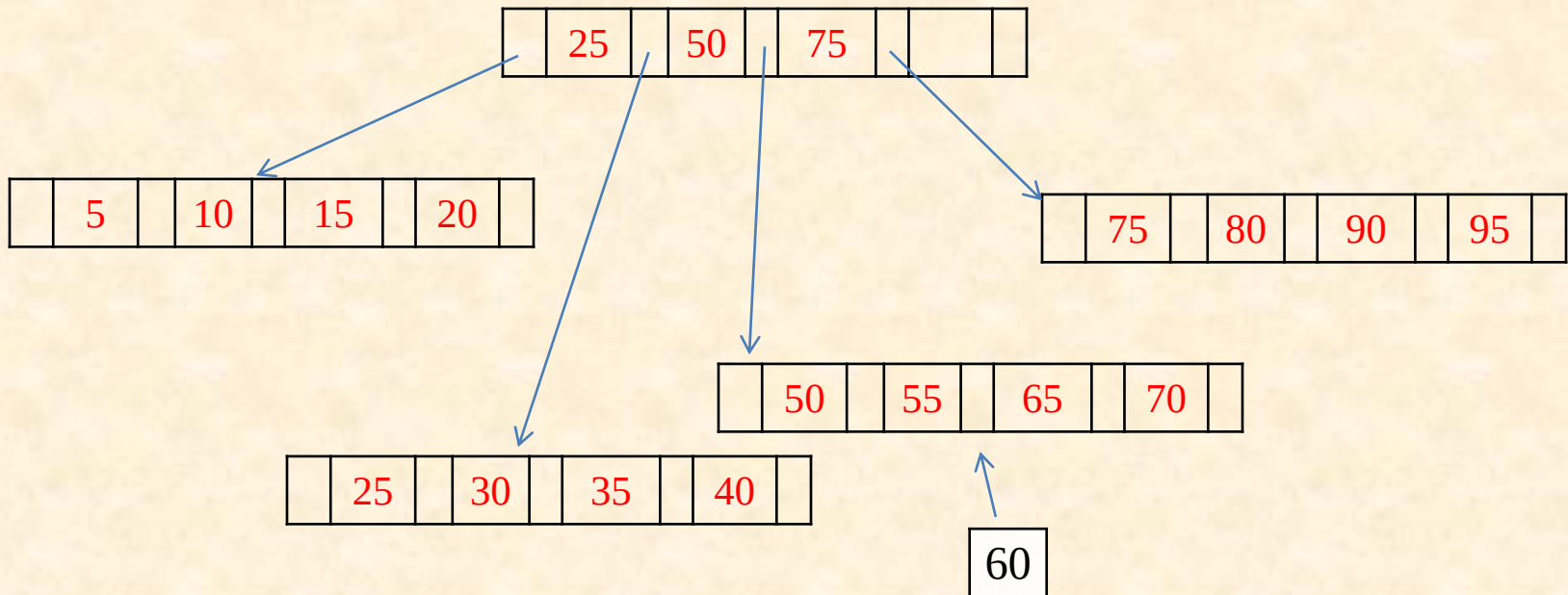
Indexing and Hashing

❑ **Example:** b) Show how the B+ tree changes as the result of each of the following operations:

- i. Insert 60
- ii. Delete 65

Solution: Insert key value 60:

60 will go to the 3rd node after 55. The node of is already full. So, split the node and insert 60.



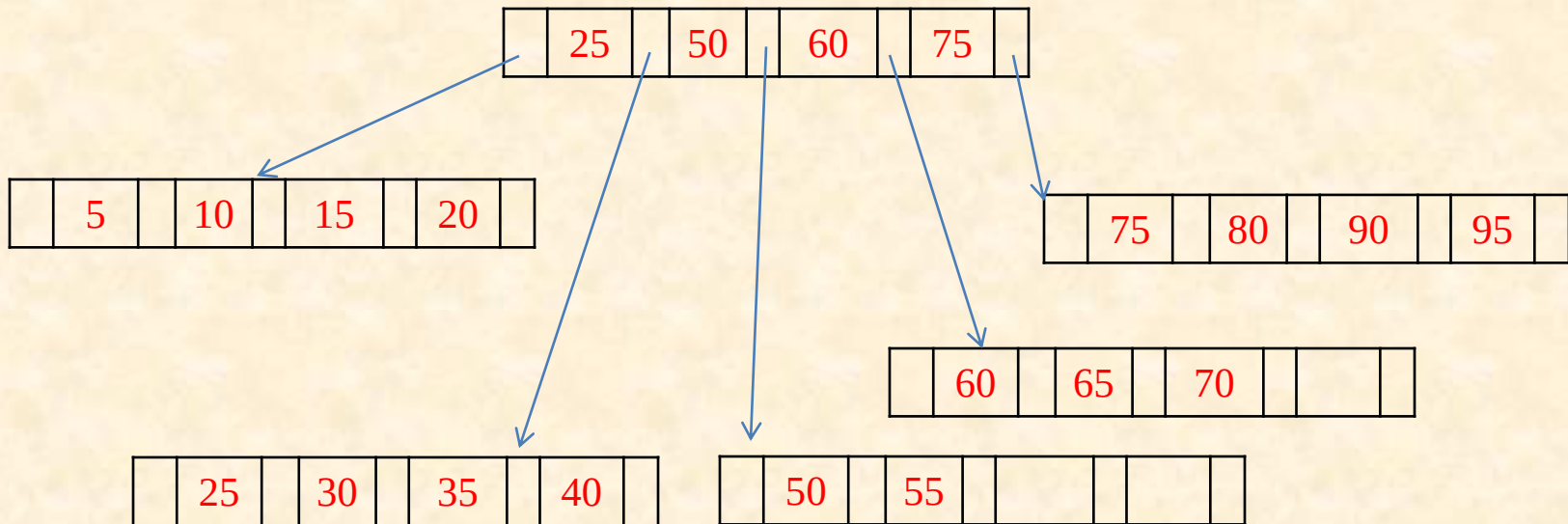


Indexing and Hashing

❑ **Example:** b) Show how the B+ tree changes as the result of each of the following operations:

- i. Insert 60 ii. Delete 65

Solution: Split the 3rd node and insert 60.



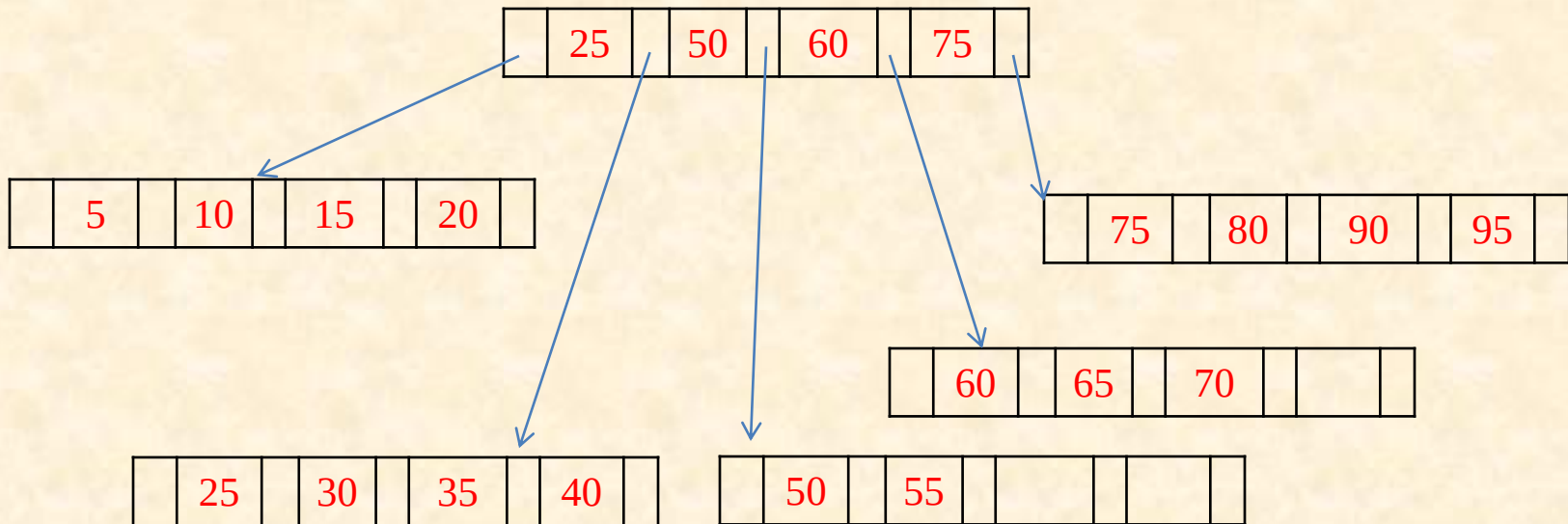


Indexing and Hashing

❑ **Example:** b) Show how the B+ tree changes as the result of each of the following operations:

- i. Insert 60 ii. Delete 65

Solution: Delete 65 from 4th node.



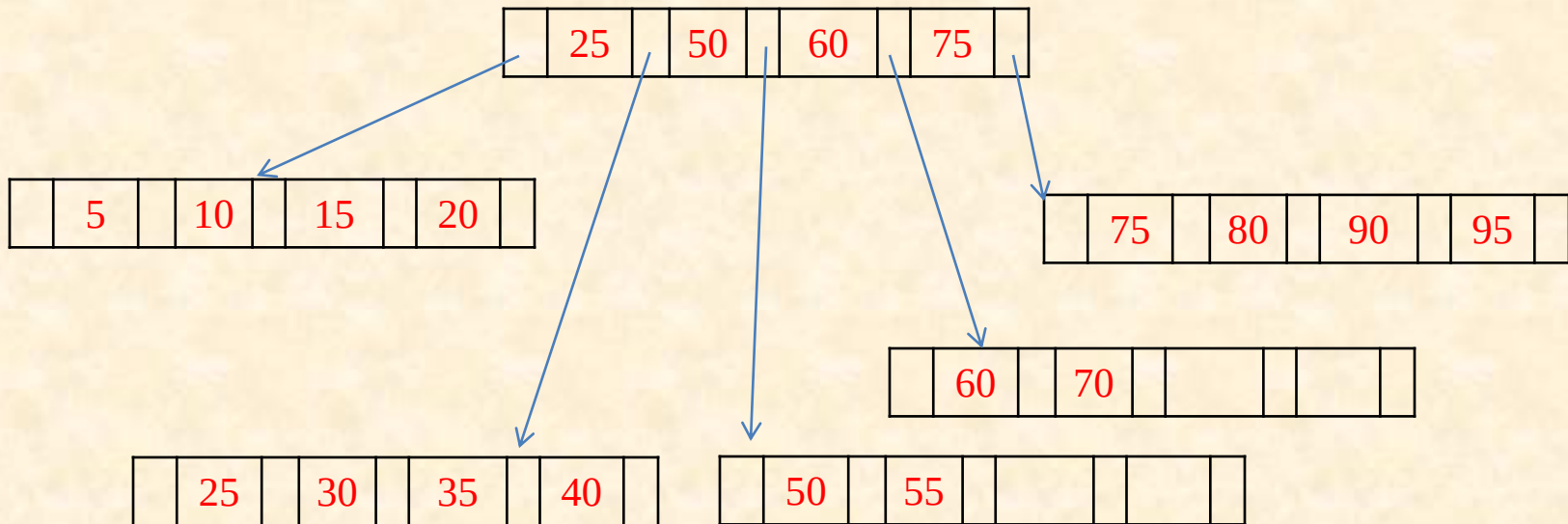


Indexing and Hashing

❑ **Example:** b) Show how the B+ tree changes as the result of each of the following operations:

- i. Insert 60 ii. Delete 65

Solution: Delete 65 from 4th node.

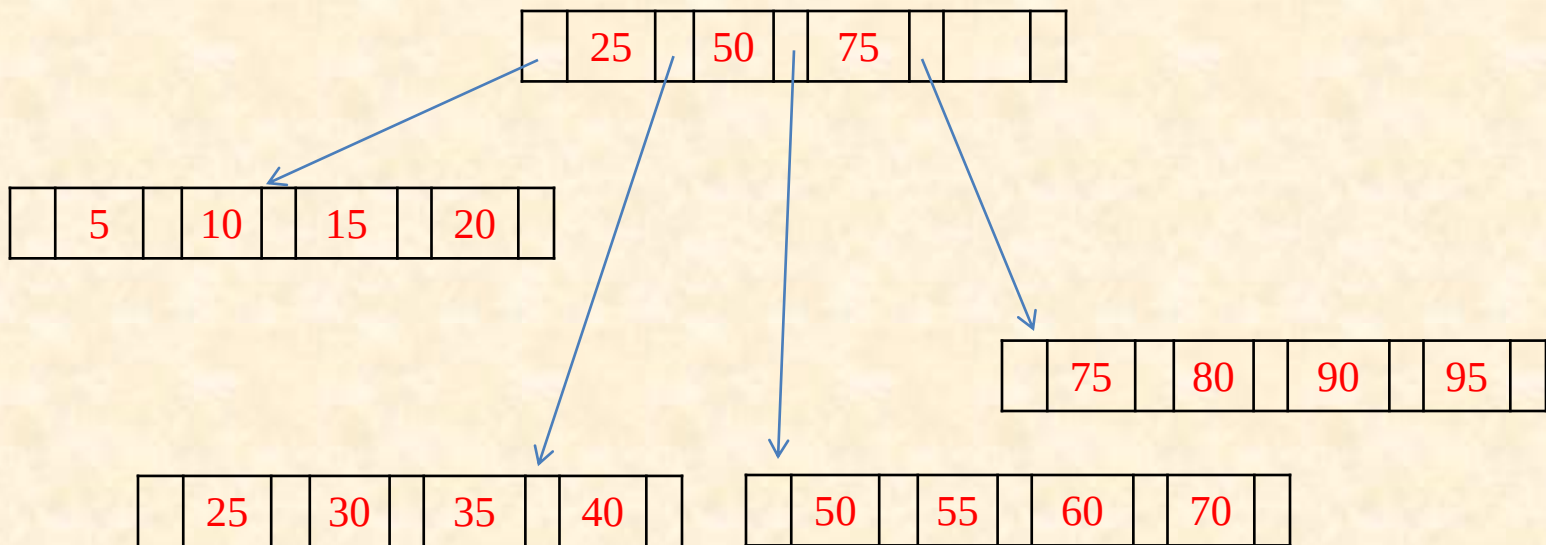




Indexing and Hashing

- ❑ **Example:** b) Show how the B+ tree changes as the result of each of the following operations:
- Insert 60
 - Delete 65

Solution: After re-arranging nodes.





Indexing and Hashing

❑ B-Tree Index Files

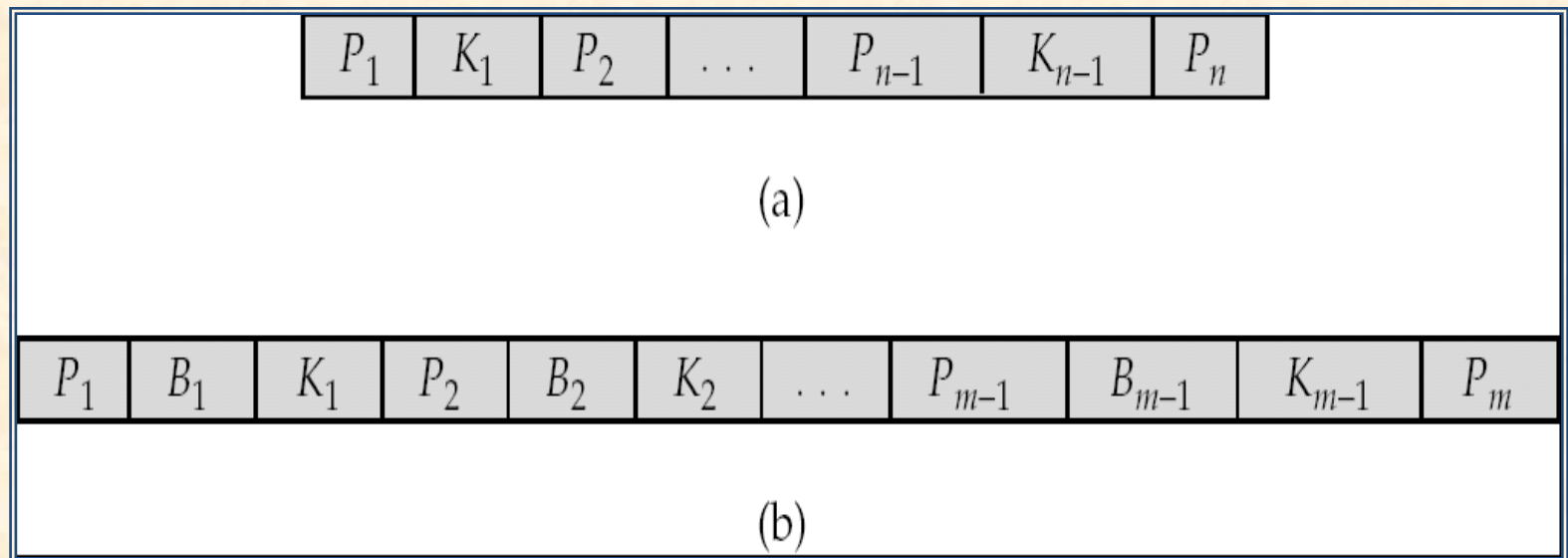
- B-tree indices are similar to B+ tree indices. The primary distinction between the two approaches is that a B-tree eliminates the redundant storage of search-key values.
- A B-tree allows search-key values to appear only once. Thus, it is necessary to include an additional pointer field for each search key in a nonleaf node. These additional pointers point to either file records or buckets for the associated search key



Indexing and Hashing

❑ B-Tree Index Files

- A generalized B-tree leaf node and a non-leaf node appear in Fig. (a) and Fig. (b) respectively.





Indexing and Hashing

❑ B-Tree Index Files

- Leaf nodes are the same as in B+ trees. In nonleaf nodes, the pointers P_i are the tree pointers that we used also for B+ trees, while the pointers B_i are bucket or file-record pointers. In the generalized B-tree in the figure, there are $n - 1$ keys in the leaf node, but there are $m - 1$ keys in the nonleaf node. This discrepancy occurs because nonleaf nodes must include pointers B_i , thus reducing the number of search keys that can be held in these nodes.

❑ Advantages of B-Tree indices

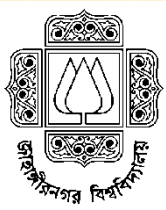
- May use less tree nodes than a corresponding B⁺ Tree.
- Sometimes possible to find search-key value before reaching leaf node.



Indexing and Hashing

❑ Disadvantages of B-Tree indices

- Only small fraction of all search-key values are found early.
- Non-leaf nodes are larger, so fan-out is reduced. Thus, B-Trees typically have greater depth than corresponding B⁺ Tree
- Insertion and deletion more complicated than in B⁺ Trees.
- Implementation is harder than B⁺ Trees.

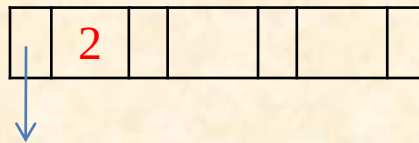


Indexing and Hashing

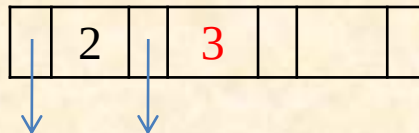
- ❑ Construct a B- tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Solution: Construction of B- tree for order $n=4$.
Search key values =3, Pointers= 4.

Insert key value 2:



Insert key value 3:



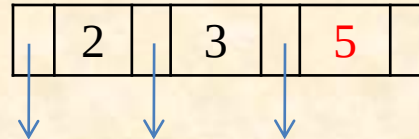


Indexing and Hashing

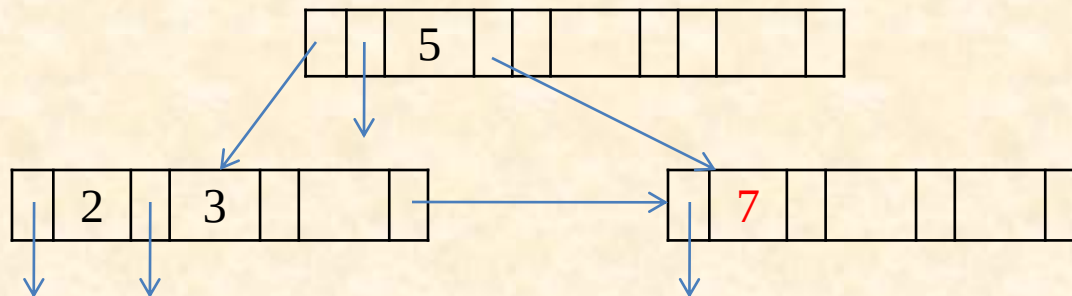
- ❑ Construct a B- tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Solution:

Insert key value 5:



Insert key value 7:



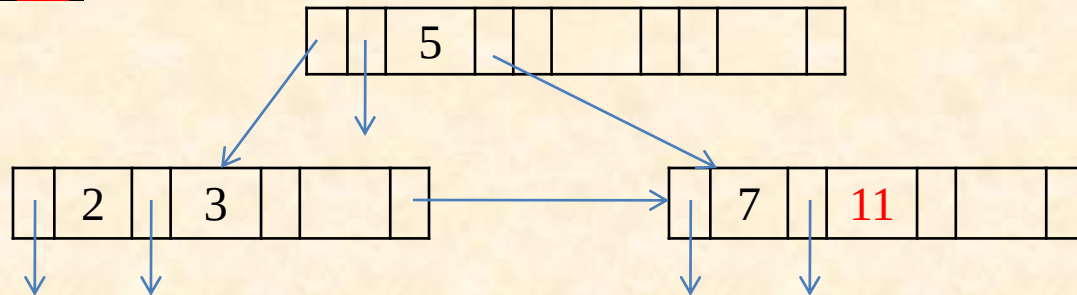


Indexing and Hashing

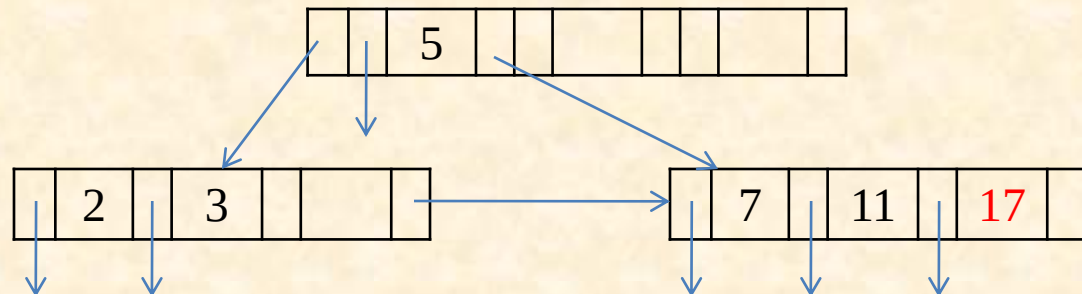
- ❑ Construct a B- tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Solution:

Insert key value 11:



Insert key value 17:



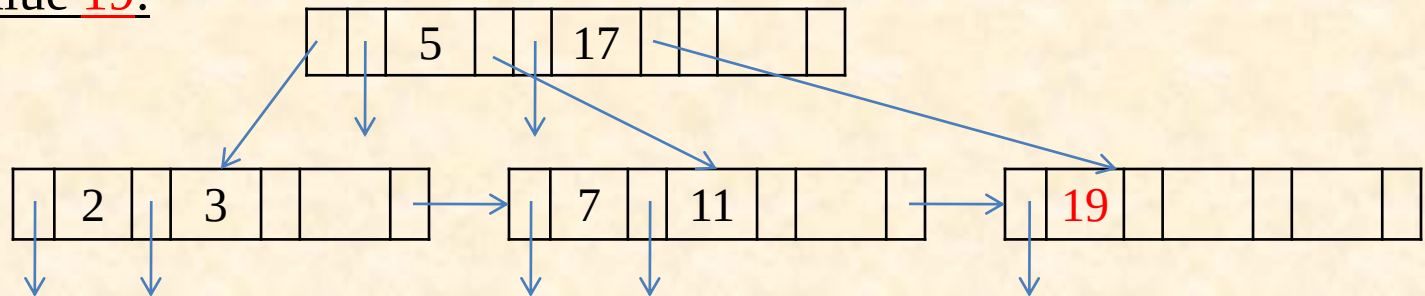


Indexing and Hashing

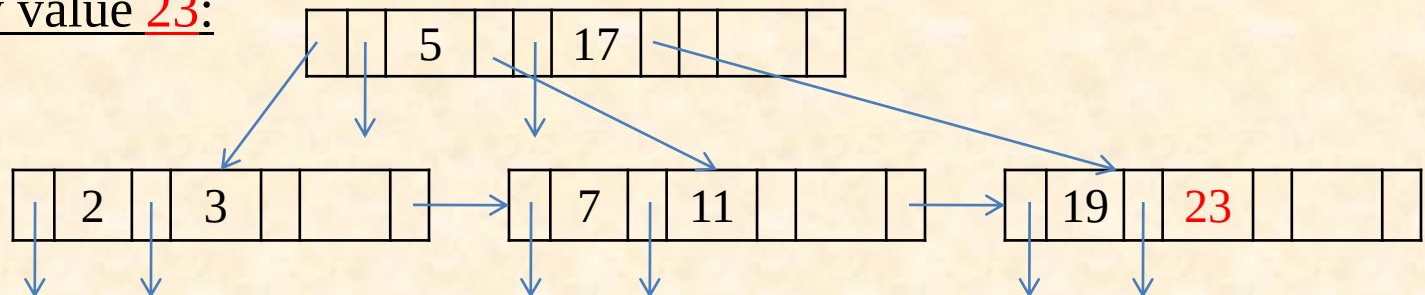
- ❑ Construct a B- tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Solution:

Insert key value 19:



Insert key value 23:



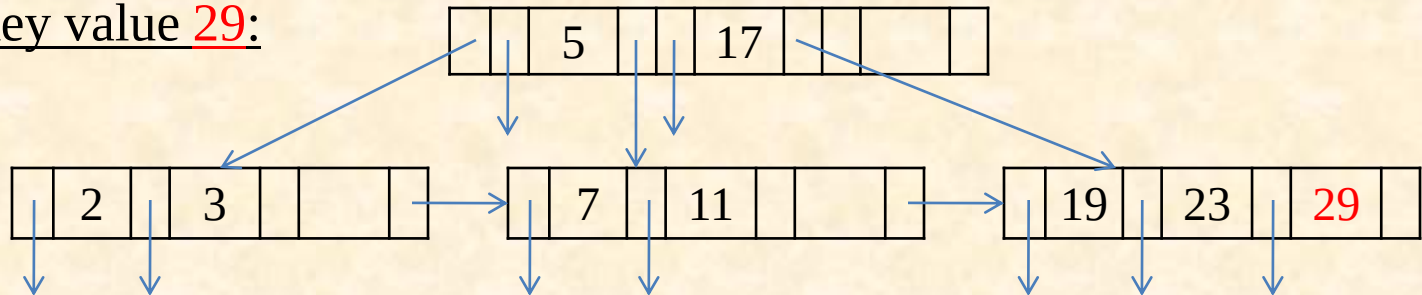


Indexing and Hashing

- ❑ Construct a B- tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Solution:

Insert key value 29:



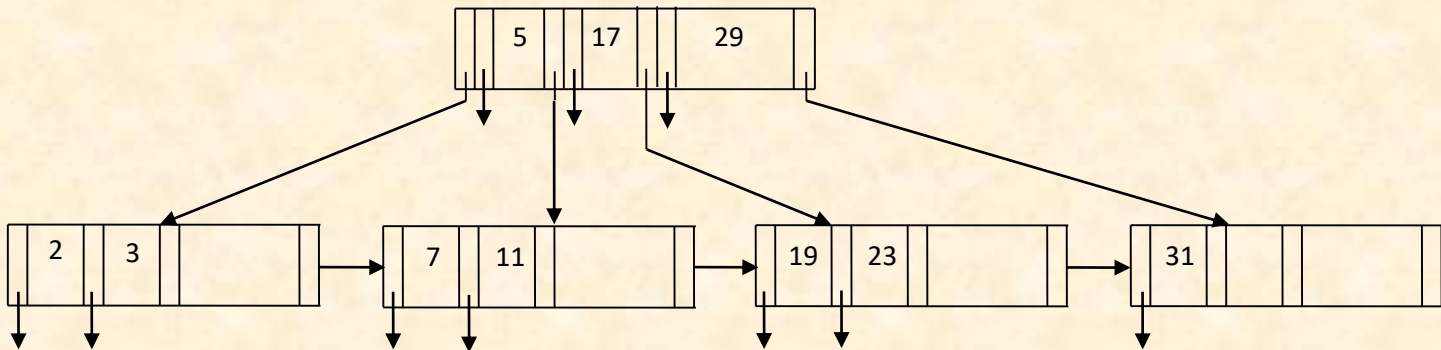


Indexing and Hashing

- ❑ Construct a B-tree for the following set of key values:
(2, 3, 5, 7, 11, 17, 19, 23, 29, 31) for $n=4$ and $n=6$.

Solution:

Insert key value 31:





Indexing and Hashing

❑ Hashing

- One disadvantage of sequential file organization is that we must use an index structure to locate data. File organizations based on the technique of hashing allow us to avoid accessing an index structure. Hashing also provides a way of constructing indices.
- File organizations based on hashing allow us to find the address of a data item directly by computing a function on the search-key value of the desired record.



Indexing and Hashing

❑ Hash File Organization

- In a hash file organization, we obtain the address of the disk block, also called the bucket containing a desired record directly by computing a function on the search-key value of the record.
- Let K denote the set of all search-key values, and let B denote the set of all bucket addresses. A hash function h is a function from K to B . Let h denote a hash function.
- To insert a record with search key K_i , we compute $h(K_i)$, which gives the address of the bucket for that record. Assume for now that there is space in the bucket to store the record. Then, the record is stored in that bucket.



Indexing and Hashing

❑ Hash File Organization

- To perform a lookup on a search-key value K_i , we simply compute $h(K_i)$, then search the bucket with that address. Suppose that two search keys, K_5 and K_7 , have the same hash value; that is, $h(K_5) = h(K_7)$. If we perform a lookup on K_5 , the bucket $h(K_5)$ contains records with search-key values K_5 and records with search key values K_7 . Thus, we have to check the search-key value of every record in the bucket to verify that the record is one that we want.



Indexing and Hashing

❑ Hash File Organization: An Example

- Let us choose a hash function for the account file using the search key **branch_name**.
- Suppose we have **26 buckets** and we define a hash function that maps names beginning with the *i*th letter of the alphabet to the *i*th bucket.
- This hash function has the virtue of simplicity, but it fails to provide a uniform distribution, since we expect more branch names to begin with such letters as B and R than Q and X.



Indexing and Hashing

❑ Hash File Organization: An Example

- Instead, we consider **10 buckets** and a hash function that computes the sum of the binary representations of the characters of a key, then returns the sum modulo the number of buckets.

- For branch name ‘Perryridge’

$$\text{Bucket no} = h(\text{Perryridge}) = 5$$

- For branch name ‘Round Hill’

$$\text{Bucket no} = h(\text{Round Hill}) = 3$$

- For branch name ‘Brighton’

$$\text{Bucket no} = h(\text{Brighton}) = 3$$



Indexing and Hashing

❑ Hash File Organization: An Example

bucket 0

--	--	--

bucket 1

--	--	--

bucket 2

--	--	--

bucket 3

Brighton	A-217	750
Round Hill	A-305	350

bucket 4

Redwood	A-222	700

bucket 5

Perryridge	A-102	400
Perryridge	A-201	900
Perryridge	A-218	700

bucket 6

--	--	--

bucket 7

Mianus	A-215	700

bucket 8

Downtown	A-101	500
Downtown	A-110	600

bucket 9

--	--	--



Indexing and Hashing

❑ Handling of Bucket Overflows

- In case of insertion, if the bucket does not have enough space, a **bucket overflow** is said to occur. Bucket overflow can occur mainly for two reasons:
- **Insufficient buckets.** The number of buckets n_B must be chosen such that $n_B > n_r/f_r$, where n_r denotes the total number of records that will be stored and f_r denotes the number of records that will fit in a bucket.
- **Skew.** Some buckets are assigned more records than are others, so a bucket may overflow even when other buckets still have space. This situation is called bucket **skew**. Skew can occur for two reasons:
 - Multiple records may have the same search key.
 - The chosen hash function may result in non-uniform distribution of search keys.



Indexing and Hashing

❑ Handling of Bucket Overflows

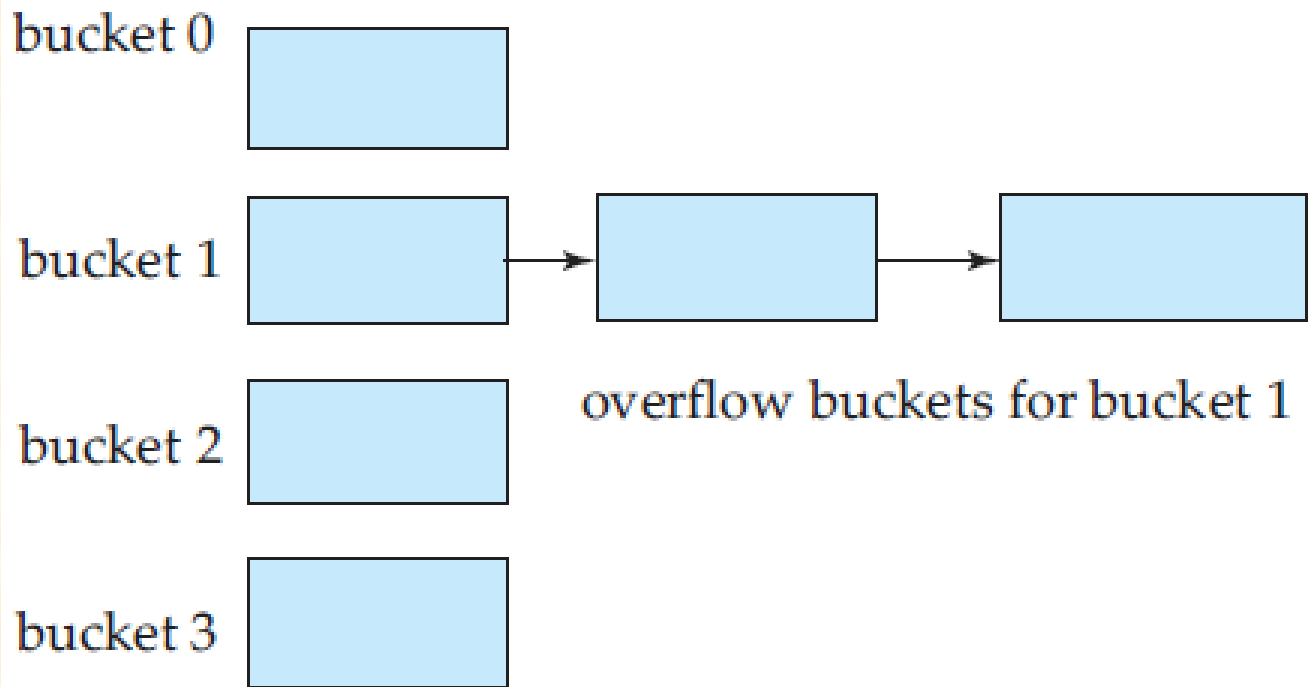
Solution:

- If a record must be inserted into a bucket b , and b is already full, the system provides an overflow bucket for b , and inserts the record into the overflow bucket. If the overflow bucket is also full, the system provides another overflow bucket, and so on. All the overflow buckets of a given bucket are chained together in a linked list.



Indexing and Hashing

❑ Handling of Bucket Overflows





Indexing and Hashing

❑ Difference between open and closed hashing

Closed Hashing:

- Closed hashing always places keys with same hash function values in same bucket (in overflow buckets also).
- If bucket is full, the system inserts records in overflow buckets.
- Different buckets can be of different sizes.
- Overflow buckets are linked together.



Indexing and Hashing

❑ Difference between open and closed hashing

Open Hashing:

- Open hashing places keys with same hash function values in different bucket if a bucket is full.
- Set of buckets is fixed there is no overflow chain
- Deletion is difficult in open hashing.



Indexing and Hashing

❑ Hash Indices

- Hashing can be used not only for file organization, but also for index-structure creation.
- We construct a hash index as follows. We apply a hash function on a search key to identify a bucket, and store the key and its associated pointers in the bucket.



Indexing and Hashing

❑ Hash Indices

